

EVALUACIÓN MODULAR 3

LUCAS AHUMADA R.
NICOLÁS JARAMILLO O.
ETHAN TRONCOSO N.

INF2223-2
Mayo 2025

ÍNDICE GENERAL

1. INTRODUCCIÓN	1
2. DOMINIO DEL PROBLEMA	2
3. ANÁLISIS	3
3.1 Requerimientos Funcionales	3
3.2 Requerimientos No Funcionales	3
3.3 Restricciones	4
3.4 Estructuras de datos utilizadas	4
3. DISEÑO	6
4.1. Arquitectura del Sistema	6
4.2. Arquitectura de Módulos	6
4.3. Herramientas y Tecnologías	7
4.4. Diagramas y Modelos Explicativos	8
5. PLANIFICACIÓN	11
5.1. Planificación grupal	11
5.2. Planificación individual	12
6. CONCLUSIÓN	13
ANEXO	14

1. INTRODUCCIÓN

La finalidad del presente informe es describir el desarrollo del sistema de gestión realizado para el parque de diversiones IBCLandia, el cual ha sido implementado en lenguaje C como parte de la Evaluación Modular 3 de Estructura de Datos. El proyecto busca representar el funcionamiento básico de un parque temático de manera digital, por lo que considera elementos como zonas, atracciones, visitantes, entradas y filas de espera, a su vez permitiendo administrar todos los datos mencionados mediante un menú diseñado para funcionar en la terminal.

Para el desarrollo del sistema se utilizaron diferentes estructuras de datos vistas durante la dictación del curso. El parque fue organizado utilizando un arreglo de zonas, donde cada zona contiene una lista doblemente enlazada de atracciones. A la vez, cada atracción contiene una fila de espera implementada mediante una lista simplemente enlazada. Las entradas del parque se almacenan en un árbol binario de búsqueda, el cual permite que sean organizadas y buscadas por medio de su ID. También se ha utilizado una lista de usuarios para poder registrar a los visitantes del parque y para que sean asociados con sus entradas y las filas a las que pertenecen.

El programa permite realizar todas las operaciones requeridas para la entrega, las cuales son: agregar, buscar, modificar, eliminar y listar todos sus datos. Entre las funcionalidades se encuentran la creación de zonas y atracciones, la administración de los visitantes, la gestión de las entradas, el ingreso de usuarios a las filas de las atracciones, la búsqueda de un usuario específico dentro de las filas y la simulación de vueltas de una atracción. Esta última funcionalidad permite representar el ciclo de uso de una atracción, llevando a los usuarios desde la fila hacia la ocupación actual de la atracción y liberando dicha ocupación cuando la vuelta termina.

A través de este proyecto desarrollado en 2 semanas de trabajo se busca aplicar de manera práctica todos los contenidos vistos en clase ya sea de estructuras estáticas, dinámicas simples y dinámicas complejas, respetando la lógica observada en los PDF disponibles en el aula virtual.

La prioridad al desarrollar este programa fue que el usuario pueda visualizar los datos de una manera intuitiva para facilitar su uso y comprensión durante la ejecución del mismo.

2. DOMINIO DEL PROBLEMA

El problema solicitado a resolver es la administración de la información principal del parque de IBCLandia. El parque necesita gestionar de manera ordenada sus zonas, atracciones, visitantes, entradas y filas de espera, ya que todos estos elementos se relacionan entre sí en el funcionamiento normal del parque y administrarlo de manera análoga sería muy complicado.

Sin un sistema organizado que utilice estructuras de datos estáticas y dinámicas como el desarrollado el manejo de estos datos podría volverse abrumador, especialmente al realizar asociaciones como registrar un visitante a una entrada, revisar el estado de una atracción o controlar una fila de espera. Por ejemplo, una atracción puede encontrarse en distintos estados los cuales influyen directamente en si los visitantes pueden o no ingresar a su fila. Del mismo modo, una entrada puede estar activa, utilizada, anulada o vencida, por lo que es necesario controlar su estado en tiempo real para evitar usos incorrectos de la misma.

Además, el parque requiere llevar un control de la ocupación de sus atracciones y zonas, lo cual conlleva una gran organización y precisión para representar correctamente a los visitantes dentro del recinto.

Abordar este problema es relevante, ya que permite aplicar los contenidos de Estructura de Datos en un caso práctico y cercano a una situación real. Este proyecto no solo permite organizar información, realizar búsquedas, modificaciones y eliminaciones, sino que también demuestra cómo las estructuras de datos pueden utilizarse para representar y resolver problemas de gestión en un entorno.

3. ANÁLISIS

En esta sección se detallan los requerimientos y el análisis del sistema de IBCLandia desarrollado para gestionar el parque de diversiones. El objetivo principal del software es poder gestionar a los usuarios, entradas, zonas, atracciones y filas del parque mediante el uso de estructura de datos implementado en ANSI C.

La captura de requerimientos se realizó considerando las principales necesidades que se necesitan para administrar un parque de diversiones, tales como registrar los visitantes, controlar las entradas, administrar las zonas y atracciones, manejar filas y poder generar reportes diarios sobre el funcionamiento del parque.

3.1 Requerimientos Funcionales

Los requerimientos funcionales son aquellas funcionalidades que debe permitir realizar el sistema al usuario. Para IBCLandia se consideraron los siguientes requerimientos:

- 1.-El sistema debe permitir registrar, buscar, modificar, eliminar y listar usuarios.
- 2.- El sistema debe permitir crear, modificar, buscar, eliminar y listar las entradas del parque.
- 3.-El sistema debe permitir asociar usuarios a entradas y controlar su ingreso o salida del parque.
- 4.- El sistema debe permitir agregar, buscar, modificar, eliminar y listar zonas del parque.
- 5.-El sistema debe permitir agregar, buscar, modificar, eliminar y listar atracciones asociadas a una zona.
- 6.- El sistema debe permitir gestionar filas de usuarios en las atracciones.
- 7.-El sistema debe permitir iniciar y terminar vueltas de atracciones, actualizando su ocupación.
- 8.-El sistema debe permitir generar reportes de ingresos, entradas vendidas, entradas usadas, visitantes, ocupación de zonas y atracciones no operativas.

3.2 Requerimientos No Funcionales

Los requerimientos no funcionales corresponden a las características de calidad que debe cumplir el sistema. Para IBCLandia, se consideran los siguientes:

1. El sistema debe ejecutarse mediante consola.
2. El sistema debe presentar menús claros y fáciles de usar.
3. El sistema debe validar datos importantes ingresados por el usuario como edad, estatura,horarios,capacidades,tipos y estados.
4. El sistema debe organizarse de forma modularizada es decir mediante funciones separadas para facilitar su comprensión y mantenimiento.
5. El sistema debe utilizar correctamente memoria dinámica, punteros y estructuras de datos.
6. El sistema debe manejar errores básicos, como datos inválidos, elementos inexistentes o punteros nulos.
7. El sistema debe responder adecuadamente en las operaciones principales de registro, búsqueda, modificación, eliminación y listado.

3.3 Restricciones

Las restricciones corresponden a las limitaciones técnicas y reglas bajo las cuales se desarrolló el sistema de IBCLandia, para este se consideraron las siguientes restricciones:

1. El sistema debe ser desarrollado en lenguaje ANSI C.
2. El sistema funciona y se utiliza sólo mediante consola, sin interfaz gráfica.
3. La información se mantiene en memoria durante la ejecución, sin base de datos ni almacenamiento permanente.
4. El parque tiene un máximo de 15 zonas.
5. Las atracciones deben estar asociadas a una zona existente.
6. Las filas deben estar asociadas a una atracción existente.
7. Un usuario debe estar dentro del parque para poder ingresar a una fila de atracción.
8. Los usuarios deben estar registrados para ser asociados a entradas o filas.
9. Una atracción debe estar operativa para permitir el ingreso de usuarios a su fila.
10. Un usuario no debe permanecer en más de una fila al mismo tiempo.
11. Las entradas poseen tipos definidos: normal, pase rápido, familiar e infantil.
12. Las entradas poseen estados definidos: activa, utilizada, anulada o vencida.
13. Los nombres de usuarios deben cumplir con el formato solicitado por el sistema.
14. Solo se permite ingresar usuarios a filas de atracciones operativas.
15. La edad y estatura de los usuarios deben encontrarse dentro de los rangos definidos por las validaciones del programa.
16. Se deben utilizar estructuras de datos como arreglos, listas enlazadas, listas doblemente enlazadas y árboles binarios de búsqueda.

3.4 Estructuras de Datos Utilizadas

El sistema almacena la información en memoria utilizando estructuras de datos en C. Cada estructura fue diseñada para representar un elemento importante del parque y sus relaciones.

Parque: La estructura Parque es la estructura principal de nuestro sistema. Desde ella se accede a las zonas, usuarios y entradas.

Contiene un arreglo de zonas, la cantidad de zonas registradas, el número de visitantes del día, una lista de usuarios y un árbol binario de búsqueda de entradas. Esta estructura funciona como el centro del sistema, ya que conecta los distintos módulos.

Arreglo estático de Zonas: Las zonas se almacenan en un arreglo de tamaño fijo. Cada posición del arreglo contiene un puntero a una zona creada dinámicamente.

Esto permite acceder directamente a las zonas mediante posiciones del arreglo y controlar la cantidad máxima de zonas permitidas. En este caso, el sistema define un máximo de 15 zonas.

Cada zona contiene sus propios datos y una lista de atracciones asociadas.

Lista Doblemente Enlazada de Atracciones: Cada zona posee una lista doblemente enlazada de atracciones que emplea nodo fantasma. Esta lista permite recorrer fácilmente las atracciones de una zona y también eliminar nodos actualizando los enlaces anterior y siguiente.

Cada nodo de la lista contiene un puntero a una atracción. Esta estructura es útil porque una zona puede tener distintas cantidades de atracciones, por lo que no se requiere un arreglo fijo para almacenarlas.

Cada atracción tiene una lista de espera asociada.

Lista Simplemente Enlazada de Fila: Cada atracción posee una fila de usuarios, implementada mediante una lista simplemente enlazada la cual cada Nodo apunta mediante un puntero hacia la estructura del usuario perteneciente a ese espacio en la fila. Esta estructura representa el orden de llegada de los usuarios que esperan para ingresar a una atracción.

Cuando se inicia una vuelta en la atracción, los usuarios se retiran desde el inicio de la fila y pasan a ocupar la atracción hasta completar su capacidad máxima. Esta estructura permite manejar filas de tamaño variable según la cantidad de usuarios que esperen.

Lista de usuarios: Los usuarios se almacenan en una lista simplemente enlazada con uso de nodo fantasma. Esta estructura permite agregar usuarios de forma dinámica sin definir previamente una cantidad máxima.

Cada nodo de la lista apunta a un usuario. Gracias a esta lista, el sistema puede recorrer todos los usuarios registrados, buscar un usuario por id, modificarlo, eliminarlo o verificar si está dentro del parque.

ABB de entradas: Las entradas se almacenan en un árbol binario de búsqueda con nodo fantasma. Cada nodo del árbol contiene una entrada y se ordena según el id de la entrada.

Esta estructura permite organizar las entradas de forma jerárquica y realizar búsquedas, inserciones, eliminaciones y listados mediante recorridos del árbol. Además, los reportes de entradas vendidas, entradas usadas e ingresos totales se calculan recorriendo este ABB.

4. DISEÑO

La solución propuesta consiste en un sistema de gestión por consola, implementado en lenguaje C, el cual modela las distintas entidades del parque temático IBCLandia mediante estructuras de datos dinámicas y estáticas. La solución es un sistema completamente interactivo mediante un menú, con funcionalidades de creación, modificación, lectura y eliminación sobre los elementos esenciales del parque temático.

4.1. Arquitectura del Sistema

El sistema se organiza en torno a la estructura central del Parque, que contiene las demás entidades. Las zonas son almacenadas en un arreglo estático de hasta 15 punteros (zonas[MAX_ZONAS]), lo que permite acceso directo por índice y compactación al eliminar. Cada zona contiene una lista doblemente enlazada de atracciones con nodo fantasma. Cada atracción contiene una lista simplemente enlazada que representa la lista de espera de dicha atracción. Los usuarios del parque se almacenan en una lista simplemente enlazada con nodo fantasma a nivel global del Parque. Las entradas se organizan en un árbol binario de búsqueda por ID, lo que permite búsquedas computacionalmente eficientes. Cada entrada tiene su propia lista de usuarios vinculados a dicha entrada por medio de punteros que los referencian, la memoria de estas estructuras de datos se gestiona dinámicamente mediante el uso de malloc, según la práctica estándar del curso.

4.2. Arquitectura de Módulos

El sistema se organiza en módulos funcionales, cada uno asociado a un menú principal. Esta separación permite ordenar el código y facilitar la mantención del programa.

Los módulos del sistema son:

1. **Menú principal:** El menú principal funciona como punto de entrada del sistema. Desde este menú, el usuario puede acceder a las distintas áreas del programa, como usuarios, entradas, zonas, atracciones, filas y reportes, este menú permite centralizar la navegación del sistema y evita que el usuario deba acceder directamente a las funciones internas del código.
2. **Menú de usuarios:** El módulo de usuarios se encarga de administrar la información de los visitantes del parque. Permite registrar nuevos usuarios, buscarlos por identificador, modificar sus datos, eliminarlos y listarlos, cada usuario posee datos como nombre, id, edad, estatura, si se encuentra actualmente dentro del parque y si estuvo previamente en el parque. Este módulo también participa en el control de ingreso y salida de visitantes.

3. **Menú de entradas:** El módulo de entradas permite administrar las entradas vendidas por el parque. Cada entrada posee un id, valor, tipo y estado, este módulo permite agregar nuevas entradas, buscar entradas existentes, modificar sus datos, cambiar su estado, eliminarlas y listar la lista. Además, permite asociar usuarios a una entrada, lo cual es útil para representar entradas individuales o familiares.
4. **Menú de zonas:** El módulo de zonas administra las distintas áreas del parque. Cada zona contiene información como nombre, código, temática, capacidad máxima, ocupación actual y horario, desde este módulo se pueden agregar, buscar, modificar, eliminar y listar zonas. Las zonas son importantes porque dentro de ellas se organizan las atracciones del parque.
5. **Menú de atracciones:** El módulo de atracciones se encarga de administrar las atracciones pertenecientes a una zona. Cada atracción posee nombre, código, capacidad máxima, ocupación actual, estado y una fila de usuarios, este módulo permite agregar atracciones a una zona específica, buscar atracciones por código, modificar sus datos, eliminar atracciones y listar las atracciones de una zona.
6. **Menú de filas:** El módulo de filas administra la espera de usuarios para ingresar a una atracción. Permite agregar usuarios a una fila, listar la fila, buscar usuarios dentro de una fila, iniciar una vuelta de atracción y terminarla, cuando se inicia una vuelta, el sistema mueve usuarios desde la fila hacia la atracción según la capacidad máxima disponible. Al terminar la vuelta, la ocupación de la atracción vuelve a cero y se actualiza la ocupación de la zona correspondiente.
7. **Menú de reportes:** El módulo de reportes permite obtener información resumida sobre el funcionamiento del parque. Entre sus funciones se encuentran calcular ingresos totales, contar entradas vendidas, contar entradas usadas, contar visitantes totales, contar visitantes dentro del parque, mostrar ocupación de zonas y listar atracciones no operativas.

4.3. Herramientas y Tecnologías

El sistema fue desarrollado en lenguaje C, compilado con GCC. Para el desarrollo se utilizó CLion como entorno de desarrollo integrado. El control de versiones fue gestionado con Git y GitHub. Las pruebas fueron realizadas manualmente desde la terminal zsh en macOS, ejecutando el programa y navegando las opciones del menú para verificar el correcto funcionamiento de cada flujo. No se necesitó el uso de bibliotecas externas, las estructuras de datos fueron implementadas desde cero, siguiendo el material del curso, utilizando únicamente las bibliotecas estándar de C (stdio.h, stdlib.h, string.h, time.h)

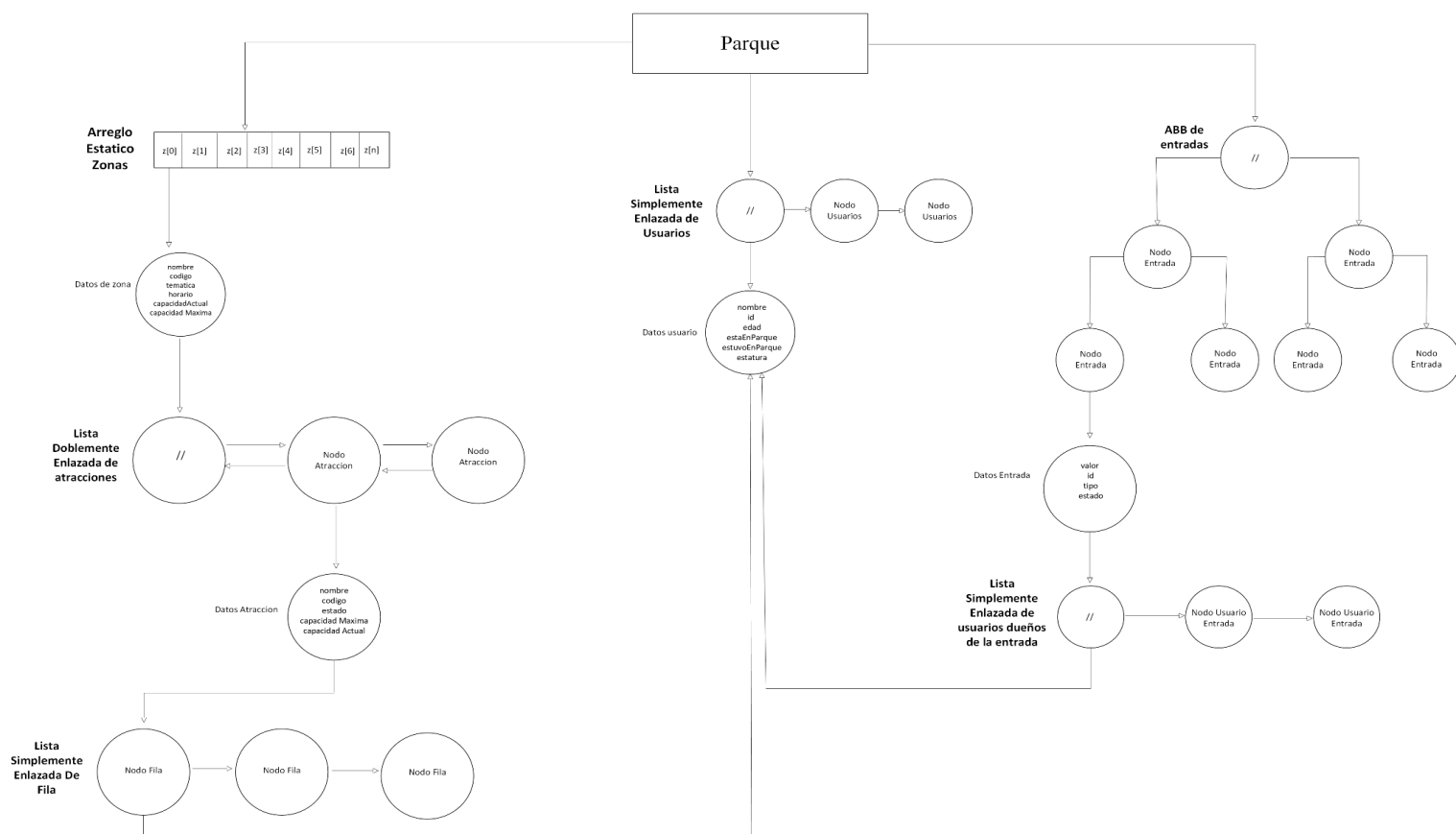
El objetivo de la solución propuesta para IBCLandia es administrar la información principal de un parque de diversiones, permitiendo gestionar usuarios, entradas, zonas, atracciones, filas y reportes.

Esta se organiza mediante una estructura principal llamada Parque, la cual centraliza los datos más importantes del sistema. Desde esta estructura se accede a las zonas del parque, la lista de usuarios registrados y el árbol de entradas. Además, el programa se divide en distintos menús para separar las funcionalidades y facilitar el uso del sistema.

4.4. Diagramas y Modelos Explicativos

Estos diagramas permiten representar gráficamente cómo se organizan los datos y cómo funcionan algunos procesos importantes del sistema.

Diagrama General de la estructura: Este diagrama realizado en Microsoft Visio muestra la organización general de los elementos del sistema. La idea es representar cómo la estructura del Parque se relaciona con las demás estructuras.



Este diagrama permite visualizar la estructura general del sistema y cómo se conectan sus principales componentes.

Diagrama de flujo de tercer nivel: El tercer nivel de anidación puede representarse como la eliminación de una atracción dentro de una zona.

Por lo tanto, el diagrama de eliminación va a mostrar el proceso para eliminar una atracción de la lista doblemente enlazada de una zona.

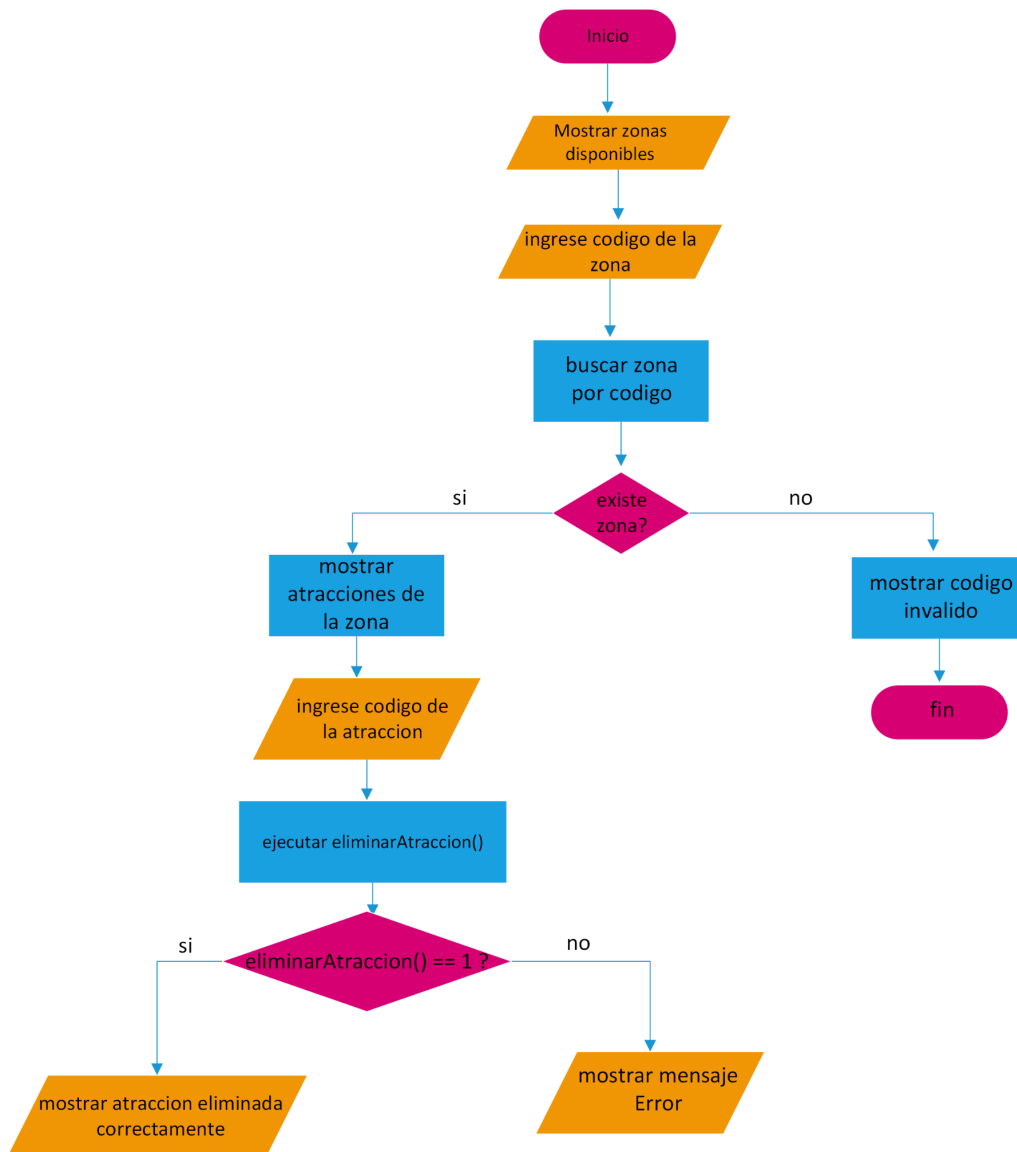
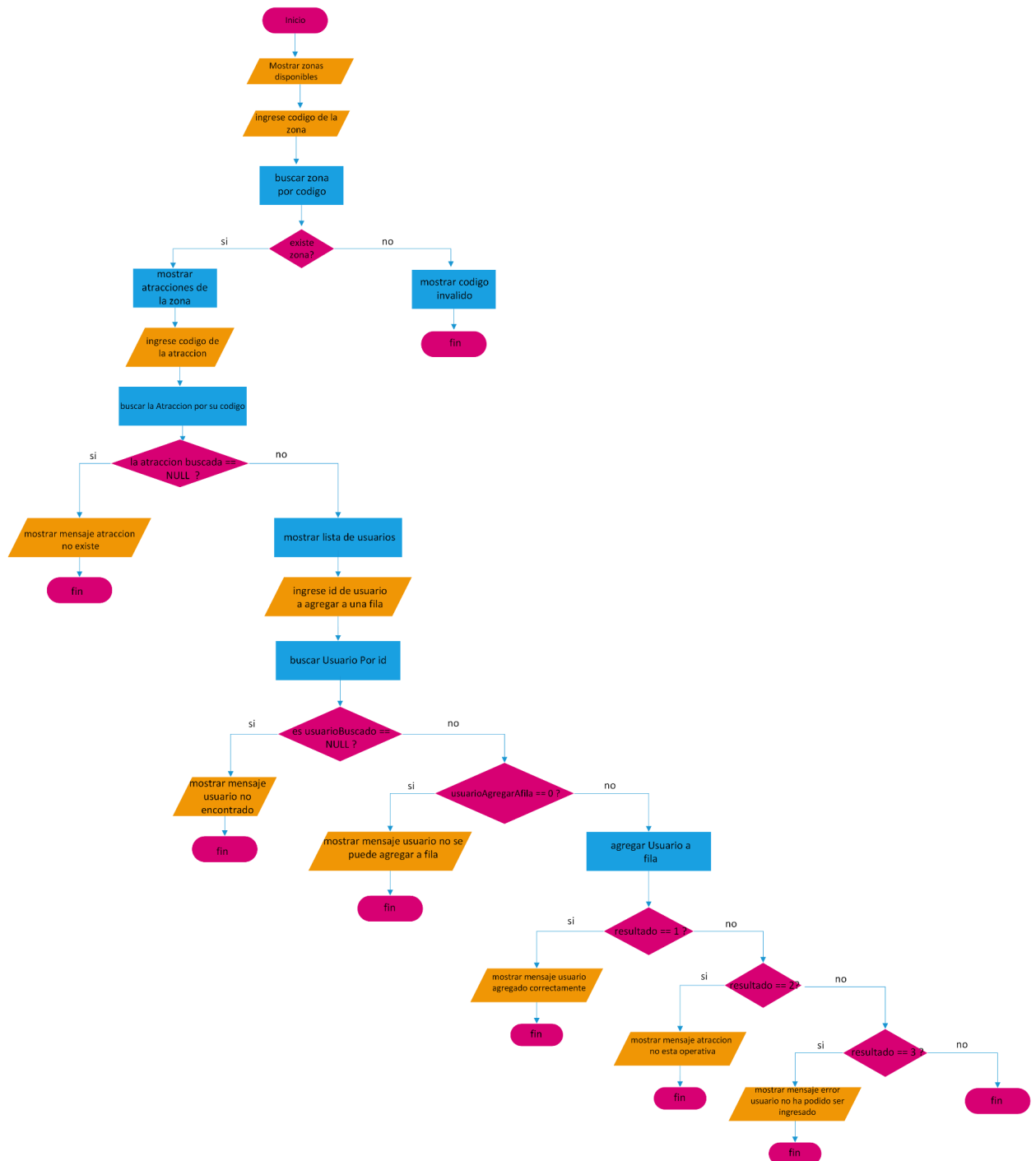


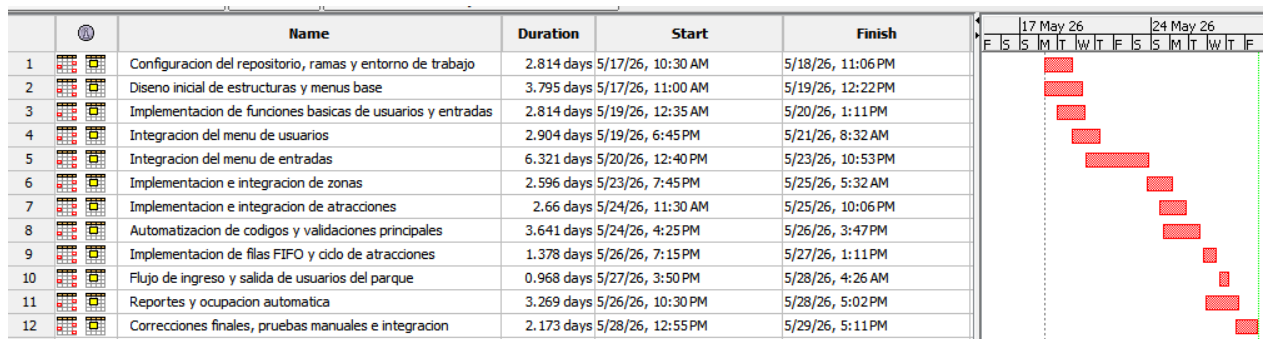
Diagrama de flujo para agregar en el último nivel: El último nivel de anidación del sistema corresponde a la fila de usuarios dentro de una atracción. Por eso, el diagrama de flujo de agregar va a representar el proceso de agregar un usuario a la fila de una atracción.



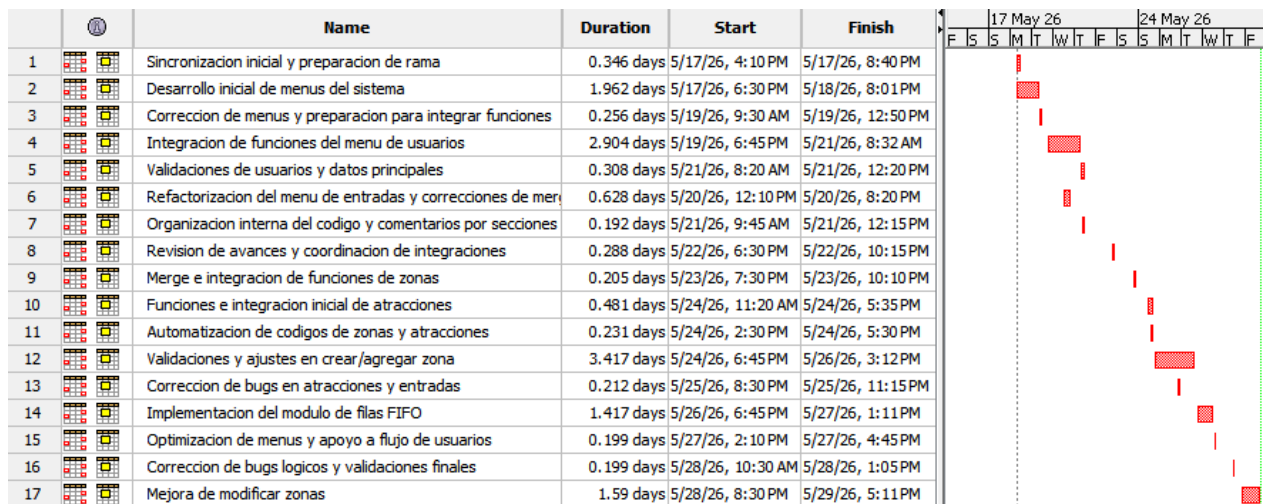
5. PLANIFICACIÓN

El proyecto fue desarrollado en un plazo de dos semanas. Donde la primera semana se dedicó al diseño de las estructuras de datos que utilizará el sistema de IBCLandia y implementación de los módulos básicos de los usuarios, zonas, parque, atracciones y entradas, completos con las funcionalidades CRUD requeridas por el sistema. En la segunda semana se implementó el módulo de filas, se agregó funcionalidad al menú con el que se opera el sistema, además de arreglar varios bugs, que impedían el correcto funcionamiento y relación entre las funciones, menús y estructuras de datos, una vez terminado el sistema se realizó una serie de pruebas simulando la correcta operación del software, finalmente se desarrolló este informe.

5.1. Planificación grupal



5.2. Planificación individual



-Nicolas Jaramillo

		Name	Duration	Start	Finish	10 May 26	17 May 26	24 May 26
						F S M T W T F S S M T W T F S S M T W T F		
1		Diseño e implementación de estructuras de datos	0.314 days	5/11/26, 11:40 AM	5/11/26, 3:45 PM			
2		crearUsuario / agregarUsuario	5.692 days	5/14/26, 8:10 PM	5/17/26, 10:13 PM			
3		buscarUsuarioPorId	2.09 days	5/17/26, 7:40 PM	5/18/26, 10:51 PM			
4		listarUsuarios / listarUsuario	0.212 days	5/19/26, 9:35 AM	5/19/26, 12:20 PM			
5		crearEntrada / agregarEntrada (ABB)	2.115 days	5/17/26, 9:30 PM	5/19/26, 1:02 AM			
6		buscarEntradaPorId / listarEntradas	1.077 days	5/18/26, 10:15 PM	5/19/26, 12:16 PM			
7		Integrar agregar, buscar y listar entradas al menú	0.205 days	5/20/26, 10:15 AM	5/20/26, 12:55 PM			
8		Integrar cambiar estado y modificar entrada	0.538 days	5/21/26, 10:45 PM	5/22/26, 5:46 AM			
9		Integrar eliminar entrada	3.628 days	5/21/26, 11:40 PM	5/23/26, 10:52 PM			
10		Integrar agregar usuario a entrada	0.205 days	5/23/26, 8:15 PM	5/23/26, 10:55 PM			
11		Refactorizar y corregir errores de compilación	8.66 days	5/20/26, 12:55 PM	5/25/26, 5:35 AM			
12		Crear función usarEntrada y validar existencia	3.135 days	5/25/26, 11:10 PM	5/27/26, 3:57 PM			
13		Validar estado activo de la entrada	3.135 days	5/25/26, 11:20 PM	5/27/26, 4:07 PM			
14		Marcar entrada como usada	3.128 days	5/25/26, 11:30 PM	5/27/26, 4:12 PM			
15		Marcar usuario como dentro del parque	0.212 days	5/27/26, 1:45 PM	5/27/26, 4:30 PM			
16		Actualizar visitantesHoy al ingresar	0.205 days	5/27/26, 2:00 PM	5/27/26, 4:40 PM			
17		Crear función registrarSalidaUsuarioParque	0.192 days	5/27/26, 2:15 PM	5/27/26, 4:45 PM			
18		Marcar usuario como fuera del parque	0.192 days	5/27/26, 2:20 PM	5/27/26, 4:50 PM			
19		Evitar que usuario entre a fila sin estar en parque	0.756 days	5/27/26, 5:15 PM	5/28/26, 3:06 AM			
20		Implementar flujo completo ingreso/salida en menú	0.365 days	5/27/26, 11:40 PM	5/28/26, 4:26 AM			
21		Pruebas flujo normal: usuario, entradas, parque	2.795 days	5/28/26, 4:30 AM	5/29/26, 4:51 PM			

-Lucas Ahumada

		Name	Duration	Start	Finish	10 May 26	17 May 26	24 May 26
						F S M T W T F S S M T W T F S S M T W T F		
1		crearZona / agregarZona	3.981 days	5/11/26, 4:30 PM	5/13/26, 8:17 PM			
2		buscarZonaPorCodigo / listarZonas	2.103 days	5/14/26, 6:00 PM	5/15/26, 9:21 PM			
3		crearAtraccion / agregarAtraccion	5.385 days	5/15/26, 9:30 PM	5/18/26, 7:33 PM			
4		listarAtracciones	1.942 days	5/18/26, 8:00 PM	5/19/26, 9:16 PM			
5		Integrar agregar y eliminar zona	2.051 days	5/20/26, 7:20 PM	5/21/26, 10:01 PM			
6		Integrar buscar, modificar y listar zonas	3.513 days	5/21/26, 10:10 PM	5/23/26, 7:52 PM			
7		Implementar contar entradas vendidas y usadas	0.199 days	5/26/26, 8:10 PM	5/26/26, 10:45 PM			
8		Implementar ingresos totales	0.199 days	5/26/26, 8:25 PM	5/26/26, 11:00 PM			
9		Calcular ocupación de zonas según sus atracciones	0.205 days	5/26/26, 8:30 PM	5/26/26, 11:10 PM			
10		Implementar visitantes dentro del parque y totales hoy	1.808 days	5/26/26, 9:00 PM	5/27/26, 8:31 PM			
11		Implementar listar atracciones no operativas	1.564 days	5/27/26, 8:40 PM	5/28/26, 5:01 PM			
12		Implementar contar personas en fila	1.551 days	5/27/26, 9:00 PM	5/28/26, 5:11 PM			
13		Integrar menú de reportes completo	0.199 days	5/28/26, 2:50 PM	5/28/26, 5:25 PM			

-Ethan Troncoso

6. CONCLUSIÓN

El desarrollo de este proyecto permitió comprender que para abordar una solución a un problema como Ingenieros Informáticos en formación no solo dependemos de programar funciones aisladas, sino de lograr que los datos realmente se relacionen correctamente entre sí. En el caso de IBCLandia, cada elemento del sistema cumple un rol importante dentro del funcionamiento del parque: las entradas permitieron controlar el acceso de los visitantes, las zonas organizar las atracciones, las filas representan el orden de espera y los reportes nos entregan información útil sobre el estado del sistema. Por eso, uno de los aprendizajes más importantes y significativos fue entender como una mala decisión en la organización de los datos a utilizar puede dificultar posteriormente la implementación de una solución.

Durante el proceso de desarrollo también se evidenció la importancia de planificar antes de programar. Al trabajar con estructuras anidadas y punteros, fue necesario definir con claridad que estructura iba a contener a otra y desde donde debía accederse a cada dato. Esto permitió evitar una solución desordenada y facilitó la integración entre los distintos módulos del sistema. Además, el trabajar en equipo obligó a mantener una lógica común entre las funciones desarrolladas por cada integrante del equipo, sobre todo al momento de fusionar las ramas del proyecto.

La experiencia adquirida al realizar este proyecto fue valiosa, ya que permitió aplicar los contenidos vistos en la asignatura en un problema más amplio que los ejercicios realizados en clases y pruebas. A diferencia de una función puntual, este sistema exige pensar en recorridos completos, validaciones, casos de error y actualizaciones de datos después de cada operación. Esto ayudó a reforzar el uso de listas enlazadas, árboles binarios de búsqueda y arreglos, pero también permitió comprender mejor la responsabilidad asociada al diseño de un programa de diseñar un programa que debe mantenerse funcionando mientras sus datos cambian constantemente.

Si bien la solución implementada cumple con los objetivos principales del proyecto, también pueden identificarse múltiples mejoras futuras, como por ejemplo, el almacenamiento permanente de los reportes, una interfaz más amigable para el usuario o una validación más estricta de los datos ingresados. Sin embargo, se considera que, dentro del contexto de la evaluación, el sistema logra representar de manera coherente y directa un funcionamiento básico de un parque como IBCLandia y demuestra cómo las EDD pueden ser utilizadas para resolver problemas de gestión de una manera directa y efectiva.

ANEXO

El código fuente es proporcionado en el aula virtual del curso INF 2223-2, en la entrega correspondiente: <https://github.com/The-Nick-J/IBCLandia>